

GUÍA DE INSTALACIÓN

Workshop fMRI preprocessing basics

Instalación local en Windows - Python 3.11 - VS Code - Jupyter



Preparado por: Luisa Fernanda Taho M

NOTA: Si ya tienes Git, y un entorno virtual con Python 3.11 **las librerías** a instalar se encuentran en la **página 11**

¿Nunca has clonado un repositorio o instalado una librería?

Puedes seguir este orden, en caso de que sí

1. Instalaremos Git, Python 3.11, Visual Studio Code y Mango.
2. Clonaremos el repositorio por interfaz o por consola.
3. Instalaremos las extensiones Python, Jupyter y NiiVue.
4. Crearemos el entorno ``.venv`` con Python 3.11.
5. Instalaremos todas las librerías del proyecto.
6. Registraremos y seleccionaremos el kernel del taller.
7. Abriremos ``.notebooks/workshop.ipynb`` y comprobaremos que todo funciona.



Canela (^_^)b

Si un comando muestra un error, nos detenemos justo ahí. No seguimos pegando comandos porque después no sabremos en qué punto se rompió la instalación.

Qué necesitamos

- Windows 10 u 11 de 64 bits.
- Conexión a internet estable: ANTsPyNet descargará dependencias y pesos.
- Al menos 15 GB libres para el entorno, la caché y los derivados.
- Acceso al repositorio de GitHub del workshop.
- Permiso para instalar programas en el computador.

(=^.^=) **Decisión del taller:** Usaremos Python 3.11 + ``.venv`` + ``.pip``. No necesitamos Conda, Miniforge, Docker, WSL ni Google Colab.

Vamos a descargar cuatro cosas

Programa	Qué descargaremos	Enlace oficial
Python	3.11.9, instalador de Windows x64	https://www.python.org/downloads/release/python-3119/
Git	Git for Windows	https://git-scm.com/download/win
VS Code	User Installer x64	https://code.visualstudio.com/download
Mango	Mango para Windows v4.1	https://mangoviewer.com/mango.html

1.1 Python 3.11

Abriremos el enlace de Python y descargaremos `Windows installer (64-bit)` de Python 3.11.9. Esta es la última versión 3.11 con el instalador clásico completo para Windows.

1. Ejecutaremos el instalador.
2. Marcaremos `Add python.exe to PATH`.
3. Elegiremos `Install Now`.
4. Al terminar, cerraremos el instalador y abriremos CMD.

Comprobación en CMD

```
py -0p
py -3.11 --version
```

Debemos ver una ruta de Python 3.11 y una versión que empiece por `Python 3.11`.

1.2 Git y Visual Studio Code

Instalaremos Git for Windows con las opciones predeterminadas. Después instalaremos VS Code usando `User Installer x64`.

Comprobación en una consola nueva

```
git --version
code --version
```

Ahora traeremos el proyecto al computador

Clonar significa descargar una copia completa del proyecto y conservar su conexión con GitHub. Podemos hacerlo de dos maneras. Elegiremos una sola.

RUTA A – INTERFAZ (recomendada)

Usaremos los botones de VS Code. Es la ruta más visual y la recomendada para este taller.

RUTA B - CONSOLA

Usaremos git clone desde CMD. Es directa y sirve para repetir el proceso sin buscar botones.

(^o^)/ URL que usaremos: <https://github.com/GNACo/Worskshop-fMRI-preprocessing-basics.git>

El nombre contiene `Worskshop` tal como aparece en GitHub. No corregiremos esa palabra dentro del comando o de la URL.



Fauna (>_<)

Si GitHub pide iniciar sesión, usaremos la cuenta que tenga acceso al repositorio. Si aparece `Repository not found`, primero revisaremos ese acceso; no cambiaremos la URL al azar.

Antes de clonar

- Crearemos o elegiremos una carpeta madre, por ejemplo `C:\Taller\_Neuroimagen`.
- No clonaremos el repositorio dentro de otra copia del mismo repositorio.
- No descargaremos el botón `Download ZIP`: queremos conservar Git.

Hagámoslo por interfaz

1. Abriremos Visual Studio Code.
2. En el panel Explorer buscaremos `Clone Repository`.
3. Pegaremos la URL del repositorio.
4. Elegiremos la carpeta madre donde queremos guardar el proyecto.
5. Cuando VS Code pregunte si deseamos abrirlo, seleccionaremos `Open`.
6. Aceptaremos `Trust` porque conocemos el origen del repositorio.

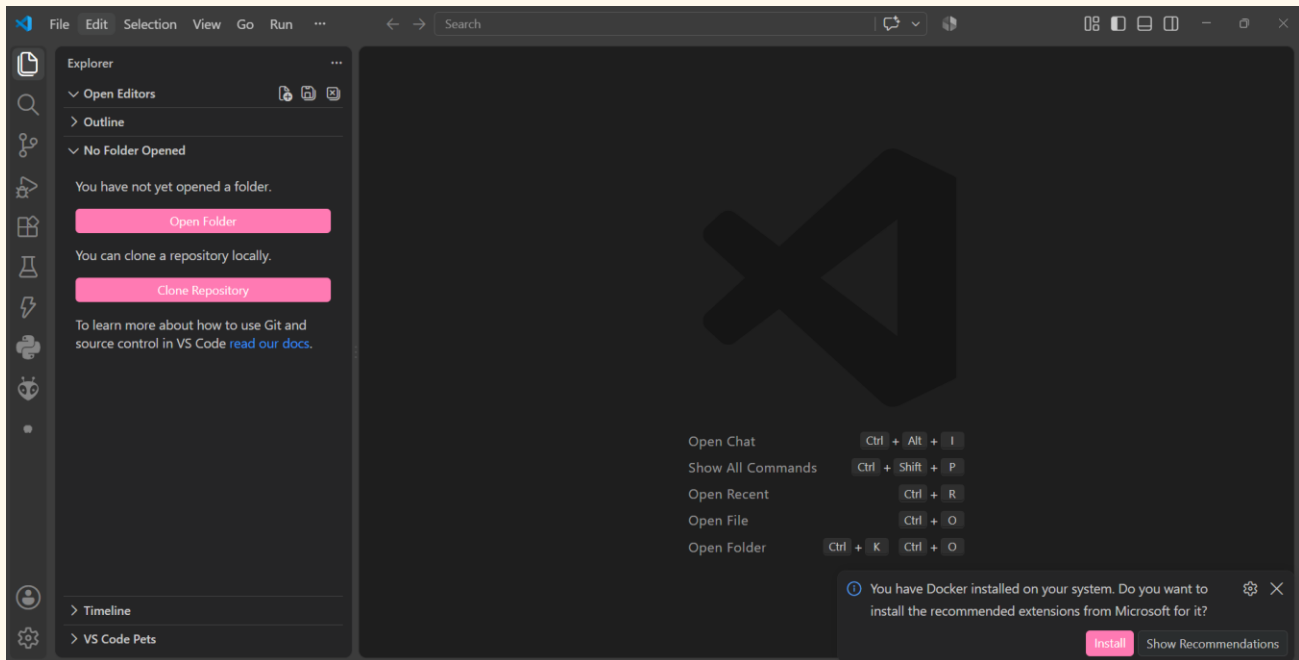


Figura 1. En la pantalla inicial de VS Code seleccionaremos Clone Repository.

(^_^) **Atajo:** También podemos abrir la paleta con `Ctrl + Shift + P`, escribir `Git: Clone` y presionar Enter.

Pegaremos la dirección exacta

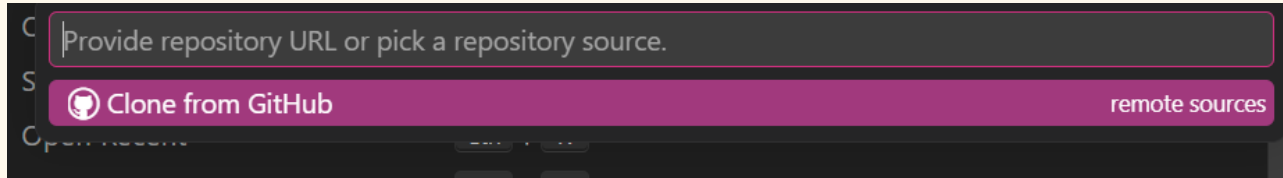


Figura 2. VS Code abre un campo en la parte superior para recibir la URL.

Esto es lo que pegaremos

```
https://github.com/GNACo/Worskshop-fMRI-preprocessing-basics.git
```

Presionaremos Enter, elegiremos `C:\Taller\_Neuroimagen` como carpeta madre y esperaremos a que termine la descarga.

Cuando termine

En Explorer debemos ver la carpeta `Worskshop-fMRI-preprocessing-basics`. Dentro de ella encontraremos, entre otras cosas:

Vista simplificada del proyecto

```
Worskshop-fMRI-preprocessing-basics/  
|-- data/  
|-- derivatives/  
|-- notebooks/  
|   |-- workshop.ipynb      <- ESTE ES EL NOTEBOOK  
|   |-- otro_notebook.ipynb  
|-- src/
```

(o_o) No abriremos el otro notebook: Durante el taller trabajaremos con `notebooks/workshop.ipynb`. El segundo notebook queda como material adicional.

Si preferimos comandos, haremos esto

Abriremos Símbolo del sistema - CMD. Crearemos una carpeta para el taller, entraremos en ella y clonaremos el repositorio.

Comandos en CMD

```
mkdir C:\Taller_Neuroimagen  
cd /d C:\Taller_Neuroimagen  
git clone https://github.com/GNACo/Worskshop-fMRI-preprocessing-basics.git  
cd Worskshop-fMRI-preprocessing-basics  
code .
```

El **punto** de ``code .`` significa: abre en VS Code la carpeta en la que estamos ahora.



Vecino del taller (^_^)b

Si elegimos esta ruta, no repetiremos luego la clonación por interfaz. A partir de aquí las dos rutas ya son exactamente iguales.

Comprobación rápida

Ejecutaremos esto dentro del repositorio

```
git status
```

Debemos ver información de una rama de Git y no el mensaje ``not a git repository``.

Ahora instalaremos tres extensiones

Extensión	Autor	Para qué la necesitamos
Python	Microsoft	Ejecutar Python y seleccionar `.venv`.
Jupyter	Microsoft	Abrir y ejecutar archivos `.ipynb`.
NiiVue	Korbinian Eckstein	Abrir `.nii` y `.nii.gz` dentro de VS Code.

Por interfaz

1. Abriremos el icono Extensions en la barra izquierda o presionaremos `Ctrl + Shift + X`.
2. Buscaremos `Python` e instalaremos la extensión de Microsoft.
3. Buscaremos `Jupyter` e instalaremos la extensión de Microsoft.
4. Buscaremos `NiiVue` e instalaremos la extensión de Korbinian Eckstein.

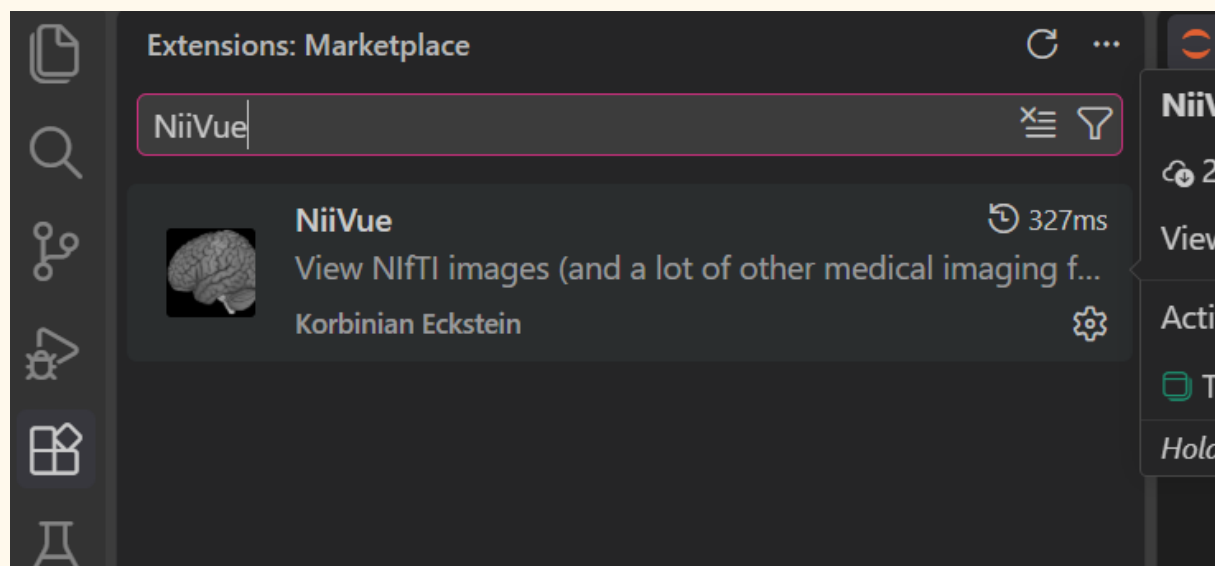


Figura 3. En Extensions buscaremos NiiVue y seleccionaremos la extensión mostrada.

Por consola

Alternativa desde CMD

```
code --install-extension ms-python.python
code --install-extension ms-toolsai.jupyter
code --install-extension KorbinianEckstein.niivue
```


NiiVue para trabajar; Mango como respaldo

NiiVue nos permite abrir las imágenes directamente dentro de VS Code. Mango es un programa independiente y nos sirve cuando queremos inspeccionar un NIFTI fuera del notebook.

Instalar Mango

1. Abriremos la página oficial de Mango.
2. Seleccionaremos `Download for Windows (v4.1)`.
3. Descargaremos y ejecutaremos el instalador de Windows.
4. Cuando termine, abriremos Mango una vez para comprobar que inicia.

Enlace oficial: <https://mangoviewer.com/mango.html>

Qué visor usaremos

- NiiVue: abrir rápido `.nii` y `.nii.gz` desde Explorer en VS Code.
- Mango: revisar imágenes, orientación y superposiciones en una ventana independiente.
- Nilearn: controles interactivos dentro del output del notebook.



Tom Nook (..)

No instalaremos Mango con `pip`. Mango es un programa aparte. NiiVue tampoco es una librería de Python: es una extensión de VS Code.

Vamos a crear `.venv` dentro del repositorio

En VS Code abriremos `Terminal > New Terminal`. Antes de escribir, confirmaremos que la terminal está ubicada en la raíz del repositorio.

En CMD, mostrará la carpeta actual

```
cd
```

La ruta debe terminar en `Worskshop-fMRI-preprocessing-basics`. Si no termina ahí, entraremos manualmente:

```
cd /d C:\Taller_Neuroimagen\Worskshop-fMRI-preprocessing-basics
```

Crear y activar

CMD

```
py -3.11 -m venv .venv
.venv\Scripts\activate
```

Cuando el entorno esté activo, veremos `(venv)` al inicio de la línea.

Comprobación

```
python --version
where python
```

La primera ruta de `where python` debe terminar en `.venv\Scripts\python.exe` y la versión debe ser 3.11.

(>_<) Si usamos PowerShell: La activación es `.venv\Scripts\Activate.ps1`. Si PowerShell bloquea el script, ejecutaremos primero `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` y volveremos a activar. El cambio solo dura en esa terminal.

Ahora sí: instalaremos todo dentro de `.venv`

Verificaremos otra vez que la terminal empiece por `.venv`. Instalaremos por bloques para reconocer de inmediato cuál grupo produce un error.

1. Herramientas de instalación

```
python -m pip install --upgrade pip setuptools wheel
```

2. Jupyter e interactividad

```
python -m pip install jupyterlab ipykernel ipywidgets ipympl
```

3. Librerías científicas y gráficas

```
python -m pip install numpy scipy pandas matplotlib seaborn plotly
```

4. Neuroimagen y estructura BIDS

```
python -m pip install nibabel nilearn pybids
```

5. ANTsPy y ANTsPyNet

```
python -m pip install antspyx antspynet
```



Tom Nook (--)

El último bloque es el más pesado. Puede tardar varios minutos y mostrar mensajes de TensorFlow. No cerraremos la terminal mientras `pip` siga trabajando.

(^_^) **Ojo con los nombres:** Instalamos `antspyx`, pero en Python escribimos `import ants`. Instalamos `pybids`, pero normalmente escribimos `from bids import BIDSLayout`.

Comprobaremos antes de abrir el notebook

Prueba general

```
python -c "import numpy, scipy, pandas, matplotlib, nibabel, nilearn, bids, ipywidgets, ipympl, plotly;
print('Librerías generales: OK')"
```

Prueba ANTs

```
python -c "import ants, antspynet; print('ANTsPy:', ants.__version__); print('ANTsPyNet: OK')"
```

Prueba Jupyter

```
jupyter lab --version
```

Si las tres pruebas terminan sin traceback, registraremos el entorno como kernel:

```
python -m ipykernel install --user --name fmri-workshop --display-name "Python 3.11 (fMRI Workshop)"
```

(^o^)/ **Resultado esperado:** Jupyter debe indicar que instaló el kernelspec `fmri-workshop`. Si ya existía, puede actualizarlo; eso no es un problema.

Una comprobación final

La lista debe incluir, entre otros, antspyx, antspynet, nibabel, nilearn, pybids y jupyterlab

```
python -m pip list
```

Trabajaremos con `notebooks/workshop.ipynb`

1. En Explorer desplegaremos la carpeta `notebooks`.
2. Abriremos `workshop.ipynb`.
3. En la esquina superior derecha seleccionaremos `Select Kernel`.
4. Elegiremos `Python Environments`.
5. Seleccionaremos `Python 3.11 (fMRI Workshop)` o el Python ubicado dentro de `.venv`.
6. Ejecutaremos la primera celda con `Shift + Enter`.

(o_o) **Notebook correcto:** En la carpeta hay dos notebooks. Para el paso a paso del taller usaremos solamente `workshop.ipynb`.

Cómo confirmar el kernel

Primera comprobación dentro del notebook

```
import sys
print(sys.version)
print(sys.executable)
```

`sys.version` debe empezar por 3.11 y `sys.executable` debe apuntar a `.venv\Scripts\python.exe`.



Canela (^_^)b

Si el notebook dice `ModuleNotFoundError`, casi siempre elegimos otro kernel. Primero revisaremos `sys.executable` antes de volver a instalar todo.

Problemas frecuentes y arreglo directo

Lo que vemos	Qué haremos
<code>`py -3.11`</code> no existe	Python 3.11 no quedó instalado. Volveremos al instalador x64 y marcaremos <code>`Add python.exe to PATH`</code> .
No aparece <code>`(.venv)`</code>	El entorno no está activo. Ejecutaremos <code>`.\venv\Scripts\activate`</code> desde la raíz del repositorio.
PowerShell bloquea <code>Activate.ps1</code>	Ejecutaremos <code>`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`</code> y activaremos otra vez.
<code>`ModuleNotFoundError`</code> en el notebook	Revisaremos el kernel. Debe apuntar a <code>`.\venv`</code> , no al Python global.
<code>`DLL load failed`</code> al importar ANTs	Instalaremos Microsoft Visual C++ Redistributable x64, reiniciaremos VS Code y repetiremos la prueba.
ANTsPyNet parece detenido	La primera ejecución puede descargar pesos. Revisaremos la terminal y esperaremos si todavía hay actividad.
El kernel se reinicia	Cerraremos otras aplicaciones, procesaremos un sujeto a la vez y reutilizaremos las salidas ya guardadas.
GitHub rechaza la clonación	Confirmaremos que la cuenta iniciada tiene acceso al repositorio y repetiremos la misma URL.

Visual C++ x64: <https://learn.microsoft.com/cpp/windows/latest-supported-vc-redist>

Checklist final

- [] Clonamos `GNACo/Worskshop-fMRI-preprocessing-basics`.
- [] Abrimos la carpeta completa del repositorio en VS Code.
- [] Instalamos las extensiones Python, Jupyter y NiiVue.
- [] Instalamos Mango como visualizador independiente.
- [] Creamos `.venv` usando Python 3.11.
- [] Activamos `.venv` y `where python` apunta al entorno.
- [] Instalamos Jupyter, librerías científicas, neuroimagen, ANTsPy y ANTsPyNet.
- [] Las pruebas de importación terminaron sin traceback.
- [] Registramos `Python 3.11 (fMRI Workshop)` como kernel.
- [] Abrimos `notebooks/workshop.ipynb`, no el otro notebook.
- [] La primera celda usa el Python de `.venv`.
- [] Podemos abrir un NIfTI con NiiVue o Mango.



Equipo del taller \(\^o\^)/

Si todas las casillas están listas, ya podemos comenzar el preprocesamiento. Ejecutaremos el notebook en orden y revisaremos el QC inmediatamente después de cada paso.

Enlaces de referencia

- Repositorio: <https://github.com/GNACo/Worskshop-fMRI-preprocessing-basics>
- Python 3.11.9: <https://www.python.org/downloads/release/python-3119/>
- VS Code: <https://code.visualstudio.com/download>
- Git for Windows: <https://git-scm.com/download/win>
- NiiVue para VS Code: <https://marketplace.visualstudio.com/items?itemName=KorbinianEckstein.niivue>
- Jupyter para VS Code: <https://marketplace.visualstudio.com/items?itemName=ms-toolsai.jupyter>
- Mango: <https://mangoviewer.com/mango.html>

Terminamos la instalación (o^_o^). Ahora sí, vamos al notebook.